Updated On : Sep-10,2020 **Time Investment :** ~30 mins

How to Create Basic Dashboard in Python with Widgets [plotly & Dash]?

Plotly has been go-to the library for data visualization by many data scientists nowadays. It provides a very easy to use API for creating interactive charts using python. Dash is another library that provides dashboard building functionality by using plotly charts. It easily integrates plotly charts into the dashboard. Apart from plotly charts it also provides various HTML tags and widgets which can be included in making the dashboard more interactive. It also lets us specify various CSS and HTML properties into components in order to change the look and feel of the dashboard. We'll be creating a simple dashboard that has few widgets to change graphs. We'll be explaining step by step process on building a basic dashboard with widgets using Plotly and Dash.

Below is a list of steps that we'll follow in order to create a dashboard using Plotly & Dash.

We'll start by importing necessary libraries.

```
import pandas as pd
import numpy as np

import plotly.express as px
import plotly.graph_objects as go
```


Load Datasets

We'll be using three datasets mentioned below of which two are easily available from the scikit-learn library.

- **IRIS Flowers Dataset** - It has flower measurements for three types of IRIS flowers. It's available from scikit-learn.
- **Wine Dataset** - It has wine ingredients measurements for three different types of wines. It's available from scikit-learn.
- **Apple OHLC Dataset** It has Apple OHLC data from Apr-2019 - Mar-2020. It can be downloaded easily as CSV from Yahoo finance.

```
iris = load_iris()

iris_df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
iris_df["FlowerType"] = [iris.target_names[target] for target in iris.target]

iris_df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	FlowerType
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

```
wine = load_wine()

wine_df = pd.DataFrame(data=wine.data, columns=wine.feature_names)
wine_df["WineType"] = [wine.target_names[target] for target in wine.target]

wine_df.head()
```

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	od280/od315_of_d
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28	2.29	5.64	1.04	
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26	1.28	4.38	1.05	
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	0.30	2.81	5.68	1.03	
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49	0.24	2.18	7.80	0.86	
4	13.24	2.59	2.87	21.0	118.0	2.80	2.69	0.39	1.82	4.32	1.04	

```
apple_df = pd.read_csv("datasets/AAPL.csv")

apple_df.head()
```

	Date	Open	High	Low	Close	Adj Close	Volume
0	2019-04-05	196.449997	197.100006	195.929993	197.000000	194.454758	18526600
1	2019-04-08	196.419998	200.229996	196.339996	200.100006	197.514709	25881700
2	2019-04-09	200.320007	202.850006	199.229996	199.500000	196.922470	35768200
3	2019-04-10	198.679993	200.740005	198.179993	200.619995	198.027985	21695300
4	2019-04-11	200.850006	201.000000	198.440002	198.949997	196.379578	20900800

Create Individual Charts

We'll first create individual charts using plotly as explained below.

1. Line Chart

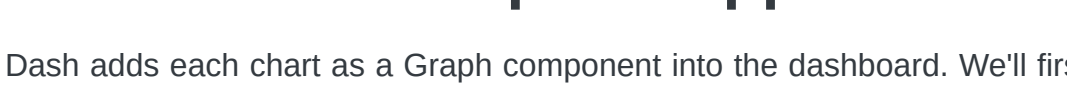
The first chart that will be included at top of the dashboard is a line chart that will plot the date on X-axis and open/high/low/close prices for that date on Y-axis.

```
chart1 = go.Figure()

chart1.add_trace(go.Scatter(x=apple_df.Date, y=apple_df.Open,
                            marker={"color":"tomato"},
                            mode="lines"))

chart1.update_layout(height=500,
                     xaxis_title="Date",
                     yaxis_title="Price ($)",
                     title="Apple Stock Prices [Apr-2019-Mar-2020]")
```

2. Scatter Plot

The second chart which will be added to the dashboard is scatter plot showing the relationship between two ingredients of wine data. All the points of the scatter plot will be color-encoded by wine type as well.

```
chart2 = px.scatter(data_frame=wine_df,
                    x=wine.feature_names[0],
                    y=wine.feature_names[1],
                    color="WineType",
                    title="%s vs %s color-encoded by wine type"%(wine.feature_names[0], wine.feature_names[1]),
                    height=500,
                    )

chart2
```



3. Bar Chart

The third chart that we'll be adding to the dashboard is a bar chart depicting the average size of flower measurement for each flower type. It'll display flower type on X-axis and average measurement on Y-axis.

```
iris_avg_by_flower_type = iris_df.groupby(by="FlowerType").mean().reset_index()

chart3 = px.bar(data_frame=iris_avg_by_flower_type,
                x="FlowerType",
                y=iris.feature_names[0],
                height=500,
                title="Avg %s Per Flower Type"%iris.feature_names[0],
                )



chart3
```

Create Dash Graph Wrapper Around Plotly Chart

Dash adds each chart as a Graph component into the dashboard. We'll first wrap each plotly chart into the `Dash Graph` component. It'll also let us give id to the chart in order to identify it as part of the dashboard's HTML component which will later include in the layout of the dashboard. The `Graph` component is available as a part of the `dash_core_components` library which gets installed when installing dash using pip/conda. Each individual HTML components like Div, H1, etc are available as a part of the `dash_html_components` library which also gets installed with a dash. We'll import it for future use.

```
import dash
import dash_core_components as dcc
import dash_html_components as html
```

```
graph1 = dcc.Graph(
        id='graph1',
        figure=chart1,
        #className="eight columns"
    )

graph2 = dcc.Graph(
        id='graph2',
        figure=chart2,
        #className="five columns"
    )

graph3 = dcc.Graph(
        id='graph3',
```

Create Individual Widgets

We'll be creating four widgets for the dashboard as mentioned below.

- **Multi-Select for Line Chart** - It'll let us select multiple values from Open/High/Low/Close prices.
- **Dropdowns for Scatter Chart** - Two dropdowns will be used to try various combinations of wine ingredients and see the relationship between them.
- **Dropdown for Bar Chart** - This dropdown will be used to select IRIS flower measurement whose average values per flower type will be displayed in a bar chart.

All widgets creation classes as available as a part of `dash_core_components`. Please find below code for each widget.

```
multi_select_line_chart = dcc.Dropdown(
        id="multi_select_line_chart",
        options=[{"value":label, "label":label} for label in ["Open", "Low", "High", "Close"]],
        value=["Open"],
        multi=True,
        clearable = False
    )
```

```
dropdown1_scatter_chart = dcc.Dropdown(
        id="dropdown1_scatter_chart",
        options=[{"value":label, "label":label} for label in wine.feature_names],
        value=wine.feature_names[0],
        className="six columns",
        clearable = False
    )

dropdown2_scatter_chart = dcc.Dropdown(
        id="dropdown2_scatter_chart",
        options=[{"value":label, "label":label} for label in wine.feature_names],
        value=wine.feature_names[1],
        className="six columns",
        clearable = False
    )
```

```
dropdown_bar_chart = dcc.Dropdown(
        id="dropdown_bar_chart",
        options=[{"value":label, "label":label} for label in iris.feature_names],
        value=iris.feature_names[0],
        clearable = False
    )
```

Create Dash Object

Dashboard created using dash will be a web app that uses flask for handling each request. We'll need to create a dash app whose layout we'll be setting in the next step by integrating all charts, widgets, and HTML components. We can create a dash app by calling the `Dash()` method from the dash library by passing it any external stylesheet if we want to give different look to the dashboard. We can also pass meta tags information to this method when creating an app.

```
external_stylesheets = ['https://codepen.io/chriddyp/pen/bWLwgP.css']

app = dash.Dash(__name__,
                external_stylesheets=external_stylesheets,
                meta_tags=[{"name": "viewport", "content": "width=device-width"}]
               )
```

Layout Charts & Widgets to Create Dashboard

Below we have included code to create the layout of the dashboard and setting that layout as the layout of the dash app created in the previous step. We have included the HTML `H2` component at the top in order to give heading to the dashboard. We have then two rows in the dashboard where the first row consists of a line chart along with its multi-select and the second row has two charts with their respective widgets. We have used HTML `Div` component in order to group things. We have combined all components into the final top `Div` which has a total dashboard.

Dashboard main components:

- **header** - Dashboard heading.
- **first row** - line chart+multi-select
- **second row** - scatter chart + dropdowns, bar chart + dropdown

We also have explicitly provided CSS properties as a part of `className` attributes of the Div component. Here property provided as a part of the `className` attribute refers to the class name defined in external CSS which was passed when creating a dash app in the previous step. It's used to set the width of Divs.

We can also explicitly specify other CSS properties not included in CSS files as a dictionary to the `style` attribute of HTML components.

```
header = html.H2(children="Simple Dashboard With Widgets")

row1 = html.Div(children=[multi_select_line_chart, graph1], className="eight columns")

scatter_div      =      html.Div(children=[html.Div(children=[dropdown1_scatter_chart,      dropdown2_scatter_chart],
className="row") , graph2], className="six columns")

bar_div = html.Div(children=[dropdown_bar_chart, graph3], className="six columns")

row2 = html.Div(children=[scatter_div, bar_div], className="eight columns")

layout = html.Div(children=[header, row1, row2], style={"text-align": "center", "justifyContent":"center"})
```

```
app.layout = layout
```

- **Input** - It accepts the id of the dashboard component and attribute of that component. It monitors changes to this attribute of the dash component and the callback function gets called when an attribute changes.
- **Output** - It accepts the id of the dashboard component and attribute of that component whose value will be changed based on changes to the `Input` component attribute.

We need to pass `Input` and `Output` as parameters of callback annotation. We need to annotate each callback function with `@app.callback` annotation passing it `Output` and `Input`. We can pass more than one `Input` as a list so that it'll monitor all those widgets and changes to any of them will result in calling this function. We need to create a function with a same number of parameters as a number of `Input` given as a list in an annotation. The function will be given new values of those widgets whose attributes are mentioned in `Input`. We can then create a new figure in function and return it. The returned figure will be set as an attribute of the `Output` figure attribute.

We have defined three callback functions below.

- **update_line** - It takes (multi-select id, value attribute) as `Input` and (line chart id, figure attribute) as `Output`.
- **update_scatter** - It takes two inputs. One Input takes (1st scatter dropdown id, value attribute) and another Input takes (2nd scatter dropdown id, value attribute). The `Output` is set to (scatter chart id, figure attribute).
- **update_bar** - It takes (bar dropdown id, value attribute) as `Input` and (bar chart id, figure attribute) as `Output`.

```
from dash.dependencies import Input, Output

@app.callback(Output('graph1', 'figure'), [Input('multi_select_line_chart', 'value')])
def update_line(price_options):
    chart1 = go.Figure()

    for price_op in price_options:
        chart1.add_trace(go.Scatter(x=apple_df.Date, y=apple_df[price_op],
                                mode="lines", name=price_op))


    chart1.update_layout(
                        xaxis_title="Date",
                        yaxis_title="Price ($)",
                        title="Apple Stock Prices [Apr-2019-Mar-2020]",
                        height=500,)
    return chart1
```

```
@app.callback(Output('graph2', 'figure'), [Input('dropdown1_scatter_chart', 'value'), Input('dropdown2_scatter_chart',
'value')])
def update_scatter(drop1, drop2):
    chart2 = px.scatter(data_frame=wine_df,
                    x=drop1,
                    y=drop2,
                    color="WineType",
                    title="%s vs %s color-encoded by wine type"%(drop1, drop2),
                    height=500,
                    )

    return chart2
```

```
@app.callback(Output('graph3', 'figure'), [Input('dropdown_bar_chart', 'value')])
def update_bar(bar_drop):
    chart3 = px.bar(data_frame=iris_avg_by_flower_type,
                x="FlowerType",
```

This ends our step by step process to build a dashboard. We'll now put the whole code together and run the dashboard.

Putting it All Together to Bring Dashboard Up

Below we have put down total code into one cell. We can save this code as `.py` python file and run it. It'll bring the dashboard up on local on port `8050` by default. We can change the default port to the port of our choice by setting port number into the `run_server()` method.

How to Create Basic Dashboard in Python with Widgets [plotly & Dash]?

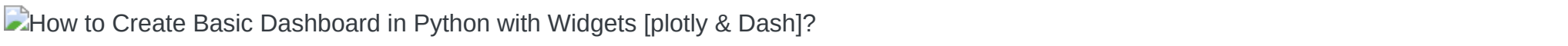
If you are interested in learning steps to deploy dashboard online then feel free to go through our another tutorial on dashboard creation using plotly/dash which explains steps to deploy dashboard on pythonanywhere.com.

- Build Dashboard using Plotly-Dash & Deploy on pythonanywhere.com

```
import pandas as pd
import numpy as np

import plotly.express as px
import plotly.graph_objects as go

from sklearn.datasets import load_iris, load_wine

import dash
import dash_core_components as dcc
import dash_html_components as html
from dash.dependencies import Input, Output


############ Loading Datasets ##################
iris = load_iris()

iris_df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
iris_df["FlowerType"] = [iris.target_names[target] for target in iris.target]

wine = load_wine()

wine_df = pd.DataFrame(data=wine.data, columns=wine.feature_names)
wine_df["WineType"] = [wine.target_names[target] for target in wine.target]


apple_df = pd.read_csv("~/datasets/AAPL.csv")


################# Line Chart ################################
chart1 = go.Figure()

chart1.add_trace(go.Scatter(x=apple_df.Date, y=apple_df.Open,
                            marker={"color":"tomato"},
                            mode="lines"))


chart1.update_layout(height=500,
                     xaxis_title="Date",
                     yaxis_title="Price ($)",
                     title="Apple Stock Prices [Apr-2019-Mar-2020]")

################# Scatter Plot ################################
chart2 = px.scatter(data_frame=wine_df,
                    x=wine.feature_names[0],
                    y=wine.feature_names[1],
                    color="WineType",
                    title="alcohol vs malic_acid color-encoded by wine type",
                    height=500,
                    )


################## Bar Chart ################################
iris_avg_by_flower_type = iris_df.groupby(by="FlowerType").mean().reset_index()

chart3 = px.bar(data_frame=iris_avg_by_flower_type,
                x="FlowerType",
                y=iris.feature_names[0],
                title="Avg %s Per Flower Type"%iris.feature_names[0],
                height=500,
                )

#################### Creating App Object ############################
external_stylesheets = ['https://codepen.io/chriddyp/pen/bWLwgP.css']
```

```
        #className="eight columns"
    )

graph2 = dcc.Graph(
        id='graph2',
        figure=chart2,
        #className="five columns"
    )

graph3 = dcc.Graph(
        id='graph3',
        figure=chart3,
        #className="five columns"
    )


############### Creating Widgets For Each Graph ##########################
multi_select_line_chart = dcc.Dropdown(
        id="multi_select_line_chart",
        options=[{"value":label, "label":label} for label in ["Open", "Low", "High", "Close"]],
        value=["Open"],
        multi=True,
        clearable = False
    )

dropdown1_scatter_chart = dcc.Dropdown(
        id="dropdown1_scatter_chart",
        options=[{"value":label, "label":label} for label in wine.feature_names],
        value=wine.feature_names[0],
        className="six columns",
        clearable = False
    )

dropdown2_scatter_chart = dcc.Dropdown(
        id="dropdown2_scatter_chart",
        options=[{"value":label, "label":label} for label in wine.feature_names],
        value=wine.feature_names[1],
        className="six columns",
        clearable = False
    )

dropdown_bar_chart = dcc.Dropdown(
        id="dropdown_bar_chart",
        options=[{"value":label, "label":label} for label in iris.feature_names],
        value=iris.feature_names[0],
        clearable = False
    )


########################## Laying out Charts & Widgets to Create App Layout ##########
header = html.H2(children="Simple Dashboard With Widgets")

row1 = html.Div(children=[multi_select_line_chart, graph1], className="eight columns")

scatter_div      =      html.Div(children=[html.Div(children=[dropdown1_scatter_chart,      dropdown2_scatter_chart],
className="row") , graph2], className="six columns")

bar_div = html.Div(children=[dropdown_bar_chart, graph3], className="six columns")

row2 = html.Div(children=[scatter_div, bar_div], className="eight columns")

layout = html.Div(children=[header, row1, row2], style={"text-align": "center", "justifyContent":"center"})

############### Setting App Layout ########################################
app.layout = layout
```

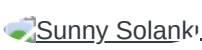
```
    for price_op in price_options:
        chart1.add_trace(go.Scatter(x=apple_df.Date, y=apple_df[price_op],
                                    mode="lines", name=price_op))


    chart1.update_layout(
                        xaxis_title="Date",
                        yaxis_title="Price ($)",
                        title="Apple Stock Prices [Apr-2019-Mar-2020]",
                        height=500,)
    return chart1



@app.callback(Output('graph2', 'figure'), [Input('dropdown1_scatter_chart', 'value'), Input('dropdown2_scatter_chart',
'value')])
def update_scatter(drop1, drop2):
    chart2 = px.scatter(data_frame=wine_df,
                    x=drop1,
                    y=drop2,
                    color="WineType",
                    title="%s vs %s color-encoded by wine type"%(drop1, drop2),
                    height=500,
                    )

    return chart2


@app.callback(Output('graph3', 'figure'), [Input('dropdown_bar_chart', 'value')])
def update_bar(bar_drop):
    chart3 = px.bar(data_frame=iris_avg_by_flower_type,
                x="FlowerType",
                y=bar_drop,
                title="Avg %s Per Flower Type"%bar_drop,
                height=500,
                )
    return chart3


################## Running App #######################################

if __name__ == "__main__":
    app.run_server(debug=True)
```



This ends our small tutorial explaining basic dashboard creation using plotly and dash with widgets. Please feel free to let us know your views in the comments section.

References

- Build Dashboard using Plotly-Dash & Deploy on pythonanywhere.com
- How to create dashboard using Python (matplotlib & panel)
- How to Create Simple Dashboard with Widgets in Python [Bokeh]
- How to Create Basic Dashboard using Streamlit and Cufflinks (Plotly)?
- Basic Dashboard using Streamlit and Matplotlib



Sunny Solanki

Comfortable Learning through Video Tutorials?

Need Help Stuck Somewhere? Need Help with Coding? Have Doubts About the Topic/Code?

When going through coding examples, it's quite common to have doubts and errors.

If you have doubts about some code examples or are stuck somewhere when trying our code, send us an email at **coderzcolumn07@gmail.com**. We'll help you or point you in the direction where you can find a solution to your problem.

You can even send us a mail if you are trying something new and need guidance regarding coding. We'll try to respond as soon as possible.

Share Views Want to Share Your Views? Have Any Suggestions?

If you want to

- provide some suggestions on topic
- share your views
- include some details in tutorial
- suggest some new topics on which we should create tutorials/blogs

Please feel free to contact us at **coderzcolumn07@gmail.com**. We appreciate and value your feedbacks. You can also support us with a small contribution by clicking **DONATE**.

dashboard, plotly-dash, widgets